

```

// This work is licensed under a Attribution-NonCommercial-
// ShareAlike 4.0 International (CC BY-NC-SA 4.0) https://
// creativecommons.org/licenses/by-nc-sa/4.0/
// © LuxAlgo

//@version=5
indicator('Smart Money Concepts [LuxAlgo]', 'LuxAlgo - Smart Money
Concepts', overlay = true, max_labels_count = 500, max_lines_count =
500, max_boxes_count = 500)
//-----}
//CONSTANTS & STRINGS & INPUTS
//-----{

BULLISH_LEG          = 1
BEARISH_LEG          = 0

BULLISH              = +1
BEARISH              = -1

GREEN                = #089981
RED                  = #F23645
BLUE                 = #2157f3
GRAY                 = #878b94
MONO_BULLISH         = #b2b5be
MONO_BEARISH         = #5d606b

HISTORICAL           = 'Historical'
PRESENT              = 'Present'

COLORED              = 'Colored'
MONOCHROME           = 'Monochrome'

ALL                  = 'All'
BOS                  = 'BOS'
CHOCH                = 'CHoCH'

TINY                 = size.tiny
SMALL                = size.small
NORMAL               = size.normal

ATR                  = 'Atr'
RANGE                = 'Cumulative Mean Range'

CLOSE                = 'Close'
HIGHLOW              = 'High/Low'

SOLID                = '_____'
DASHED               = '-----'
DOTTED               = '.....'

SMART_GROUP          = 'Smart Money Concepts'
INTERNAL_GROUP        = 'Real Time Internal Structure'
SWING_GROUP           = 'Real Time Swing Structure'

```

```

BLOCKS_GROUP                = 'Order Blocks'
EQUAL_GROUP                 = 'EQH/EQL'
GAPS_GROUP                  = 'Fair Value Gaps'
LEVELS_GROUP                = 'Highs & Lows MTF'
ZONES_GROUP                 = 'Premium & Discount Zones'

modeTooltip                  = 'Allows to display historical
Structure or only the recent ones'
styleTooltip                 = 'Indicator color theme'
showTrendTooltip             = 'Display additional candles with a
color reflecting the current trend detected by structure'
showInternalsTooltip         = 'Display internal market
structure'
internalFilterConfluenceTooltip = 'Filter non significant internal
structure breakouts'
showStructureTooltip         = 'Display swing market Structure'
showSwingsTooltip            = 'Display swing point as labels on
the chart'
showHighLowSwingsTooltip     = 'Highlight most recent strong and
weak high/low points on the chart'
showInternalOrderBlocksTooltip = 'Display internal order blocks on
the chart\n\nNumber of internal order blocks to display on the
chart'
showSwingOrderBlocksTooltip  = 'Display swing order blocks on the
chart\n\nNumber of internal swing blocks to display on the chart'
orderBlockFilterTooltip      = 'Method used to filter out
volatile order blocks \n\nIt is recommended to use the cumulative
mean range method when a low amount of data is available'
orderBlockMitigationTooltip  = 'Select what values to use for
order block mitigation'
showEqualHighsLowsTooltip    = 'Display equal highs and equal
lows on the chart'
equalHighsLowsLengthTooltip  = 'Number of bars used to confirm
equal highs and equal lows'
equalHighsLowsThresholdTooltip = 'Sensitivity threshold in a range
(0, 1) used for the detection of equal highs & lows\n\nLower values
will return fewer but more pertinent results'
showFairValueGapsTooltip     = 'Display fair values gaps on the
chart'
fairValueGapsThresholdTooltip = 'Filter out non significant fair
value gaps'
fairValueGapsTimeframeTooltip = 'Fair value gaps timeframe'
fairValueGapsExtendTooltip   = 'Determine how many bars to extend
the Fair Value Gap boxes on chart'
showPremiumDiscountZonesTooltip = 'Display premium, discount, and
equilibrium zones on chart'

modeInput                    = input.string( HISTORICAL, 'Mode',
group = SMART_GROUP,        tooltip = modeTooltip, options =
[HISTORICAL, PRESENT])
styleInput                    = input.string( COLORED,      'Style',
group = SMART_GROUP,        tooltip = styleTooltip, options = [COLORED,
MONOCHROME])
showTrendInput                = input(           false,      'Color

```

```

Candles',          group = SMART_GROUP,    tooltip =
showTrendTooltip)

showInternalsInput      = input(          true,      'Show
Internal Structure',    group = INTERNAL_GROUP, tooltip =
showInternalsTooltip)
showInternalBullInput   = input.string( ALL,      'Bullish
Structure',             group = INTERNAL_GROUP, inline = 'ibull', options
= [ALL,BOS,CHOCH])
internalBullColorInput  = input(          GREEN,     '',
group = INTERNAL_GROUP, inline = 'ibull')
showInternalBearInput   = input.string( ALL,      'Bearish
Structure' ,            group = INTERNAL_GROUP, inline = 'ibear', options
= [ALL,BOS,CHOCH])
internalBearColorInput  = input(          RED,       '',
group = INTERNAL_GROUP, inline = 'ibear')
internalFilterConfluenceInput = input(          false,
'Confluence Filter',    group = INTERNAL_GROUP, tooltip =
internalFilterConfluenceTooltip)
internalStructureSize   = input.string( TINY,
'Internal Label Size',  group = INTERNAL_GROUP, options =
[TINY,SMALL,NORMAL])

showStructureInput      = input(          true,      'Show
Swing Structure',       group = SWING_GROUP,    tooltip =
showStructureTooltip)
showSwingBullInput      = input.string( ALL,      'Bullish
Structure',             group = SWING_GROUP,    inline = 'bull',
options = [ALL,BOS,CHOCH])
swingBullColorInput     = input(          GREEN,     '',
group = SWING_GROUP,    inline = 'bull')
showSwingBearInput      = input.string( ALL,      'Bearish
Structure',             group = SWING_GROUP,    inline = 'bear',
options = [ALL,BOS,CHOCH])
swingBearColorInput     = input(          RED,       '',
group = SWING_GROUP,    inline = 'bear')
swingStructureSize      = input.string( SMALL,     'Swing
Label Size',            group = SWING_GROUP,    options =
[TINY,SMALL,NORMAL])
showSwingsInput         = input(          false,     'Show
Swings Points',         group = SWING_GROUP,    tooltip =
showSwingsTooltip,inline = 'swings')
swingsLengthInput       = input.int(      50,        '',
group = SWING_GROUP,    minval = 10,            inline =
'swings')
showHighLowSwingsInput  = input(          true,      'Show
Strong/Weak High/Low', group = SWING_GROUP,    tooltip =
showHighLowSwingsTooltip)

showInternalOrderBlocksInput = input(          true,
'Internal Order Blocks' , group = BLOCKS_GROUP, tooltip =
showInternalOrderBlocksTooltip, inline = 'iob')
internalOrderBlocksSizeInput = input.int(  5,        '',
group = BLOCKS_GROUP,    minval = 1, maxval = 20,

```

```

inline = 'iob')
showSwingOrderBlocksInput      = input(          false,      'Swing
Order Blocks',                group = BLOCKS_GROUP,  tooltip =
showSwingOrderBlocksTooltip,   inline = 'ob')
swingOrderBlocksSizeInput      = input.int(      5,          '',
group = BLOCKS_GROUP,  minval = 1, maxval = 20,
inline = 'ob')
orderBlockFilterInput          = input.string( 'Atr',        'Order
Block Filter',                group = BLOCKS_GROUP,  tooltip =
orderBlockFilterTooltip,       options = [ATR, RANGE])
orderBlockMitigationInput      = input.string( HIGHLOW,      'Order
Block Mitigation',           group = BLOCKS_GROUP,  tooltip =
orderBlockMitigationTooltip,   options = [CLOSE,HIGHLOW])
internalBullishOrderBlockColor = input.color(color.new(#3179f5,
80), 'Internal Bullish OB',   group = BLOCKS_GROUP)
internalBearishOrderBlockColor = input.color(color.new(#f77c80,
80), 'Internal Bearish OB',   group = BLOCKS_GROUP)
swingBullishOrderBlockColor    = input.color(color.new(#1848cc,
80), 'Bullish OB',            group = BLOCKS_GROUP)
swingBearishOrderBlockColor    = input.color(color.new(#b22833,
80), 'Bearish OB',            group = BLOCKS_GROUP)

showEqualHighsLowsInput        = input(          true,        'Equal
High/Low',                    group = EQUAL_GROUP,  tooltip =
showEqualHighsLowsTooltip)
equalHighsLowsLengthInput      = input.int(      3,          'Bars
Confirmation',                group = EQUAL_GROUP,  tooltip =
equalHighsLowsLengthTooltip,   minval = 1)
equalHighsLowsThresholdInput   = input.float(    0.1,
'Threshold',                  group = EQUAL_GROUP,  tooltip =
equalHighsLowsThresholdTooltip, minval = 0, maxval = 0.5, step =
0.1)
equalHighsLowsSizeInput        = input.string( TINY,        'Label
Size',                        group = EQUAL_GROUP,  options =
[TINY,SMALL,NORMAL])

showFairValueGapsInput         = input(          false,      'Fair
Value Gaps',                  group = GAPS_GROUP,   tooltip =
showFairValueGapsTooltip)
fairValueGapsThresholdInput     = input(          true,        'Auto
Threshold',                   group = GAPS_GROUP,   tooltip =
fairValueGapsThresholdTooltip)
fairValueGapsTimeframeInput     = input.timeframe('',
'Timeframe',                  group = GAPS_GROUP,   tooltip =
fairValueGapsTimeframeTooltip)
fairValueGapsBullColorInput     = input.color(color.new(#00ff68,
70), 'Bullish FVG' , group = GAPS_GROUP)
fairValueGapsBearColorInput     = input.color(color.new(#ff0008,
70), 'Bearish FVG' , group = GAPS_GROUP)
fairValueGapsExtendInput        = input.int(      1,          'Extend
FVG',                          group = GAPS_GROUP,   tooltip =
fairValueGapsExtendTooltip,    minval = 0)

showDailyLevelsInput           = input(          false,      'Daily',

```

```

group = LEVELS_GROUP,    inline = 'daily')
dailyLevelsStyleInput    = input.string( SOLID,      '',
group = LEVELS_GROUP,    inline = 'daily',    options =
[SOLID,DASHED,DOTTED])
dailyLevelsColorInput    = input(                BLUE,      '',
group = LEVELS_GROUP,    inline = 'daily')
showWeeklyLevelsInput    = input(                false,
'Weekly',    group = LEVELS_GROUP,    inline = 'weekly')
weeklyLevelsStyleInput    = input.string( SOLID,      '',
group = LEVELS_GROUP,    inline = 'weekly',    options =
[SOLID,DASHED,DOTTED])
weeklyLevelsColorInput    = input(                BLUE,      '',
group = LEVELS_GROUP,    inline = 'weekly')
showMonthlyLevelsInput    = input(                false,
'Monthly',    group = LEVELS_GROUP,    inline = 'monthly')
monthlyLevelsStyleInput    = input.string( SOLID,      '',
group = LEVELS_GROUP,    inline = 'monthly',    options =
[SOLID,DASHED,DOTTED])
monthlyLevelsColorInput    = input(                BLUE,      '',
group = LEVELS_GROUP,    inline = 'monthly')

showPremiumDiscountZonesInput    = input(                false,
'Premium/Discount Zones',    group = ZONES_GROUP , tooltip =
showPremiumDiscountZonesTooltip)
premiumZoneColorInput    = input.color(    RED,          'Premium
Zone',    group = ZONES_GROUP)
equilibriumZoneColorInput    = input.color(    GRAY,
'Equilibrium Zone',    group = ZONES_GROUP)
discountZoneColorInput    = input.color(    GREEN,
'Discount Zone',    group = ZONES_GROUP)

//-----
//-----}
//DATA STRUCTURES & VARIABLES
//-----
//-----{
// @type                                UDT representing alerts as bool
fields
// @field internalBullishBOS            internal structure custom alert
// @field internalBearishBOS            internal structure custom alert
// @field internalBullishCHoCH            internal structure custom alert
// @field internalBearishCHoCH            internal structure custom alert
// @field swingBullishBOS                swing structure custom alert
// @field swingBearishBOS                swing structure custom alert
// @field swingBullishCHoCH            swing structure custom alert
// @field swingBearishCHoCH            swing structure custom alert
// @field internalBullishOrderBlock    internal order block custom
alert
// @field internalBearishOrderBlock    internal order block custom
alert
// @field swingBullishOrderBlock        swing order block custom alert
// @field swingBearishOrderBlock        swing order block custom alert
// @field equalHighs                    equal high low custom alert
// @field equalLows                    equal high low custom alert

```

// @field bullishFairValueGap	fair value gap custom alert
// @field bearishFairValueGap	fair value gap custom alert
type alerts	
bool internalBullishBOS	= false
bool internalBearishBOS	= false
bool internalBullishCHoCH	= false
bool internalBearishCHoCH	= false
bool swingBullishBOS	= false
bool swingBearishBOS	= false
bool swingBullishCHoCH	= false
bool swingBearishCHoCH	= false
bool internalBullishOrderBlock	= false
bool internalBearishOrderBlock	= false
bool swingBullishOrderBlock	= false
bool swingBearishOrderBlock	= false
bool equalHighs	= false
bool equalLows	= false
bool bullishFairValueGap	= false
bool bearishFairValueGap	= false
// @type	UDT representing last swing
extremes (top & bottom)	
// @field top	last top swing price
// @field bottom	last bottom swing price
// @field barTime	last swing bar time
// @field barIndex	last swing bar index
// @field lastTopTime	last top swing time
// @field lastBottomTime	last bottom swing time
type trailingExtremes	
float top	
float bottom	
int barTime	
int barIndex	
int lastTopTime	
int lastBottomTime	
// @type	UDT representing Fair Value Gaps
// @field top	top price
// @field bottom	bottom price
// @field bias	bias (BULLISH or BEARISH)
// @field topBox	top box
// @field bottomBox	bottom box
type fairValueGap	
float top	
float bottom	
int bias	
box topBox	
box bottomBox	
// @type	UDT representing trend bias
// @field bias	BULLISH or BEARISH
type trend	
int bias	

<pre>// @type Lows display // @field l_line // @field l_label type equalDisplay line l_line = na label l_label = na // @type (swing point) // @field currentLevel // @field lastLevel // @field crossed // @field barTime // @field barIndex type pivot float currentLevel float lastLevel bool crossed int barTime = time int barIndex = bar_index // @type // @field barHigh // @field barLow // @field barTime // @field bias type orderBlock float barHigh float barLow int barTime int bias // @variable var pivot swingHigh // @variable var pivot swingLow // @variable var pivot internalHigh // @variable var pivot internalLow // @variable var pivot equalHigh // @variable var pivot equalLow // @variable var trend swingTrend // @variable var trend internalTrend // @variable var equalDisplay equalHighDisplay // @variable var equalDisplay equalLowDisplay // @variable</pre>	<pre>UDT representing Equal Highs displayed line displayed label UDT representing a pivot point current price level last price level true if price level is crossed bar time bar index UDT representing an order block bar high bar low bar time BULLISH or BEARISH current swing pivot high = pivot.new(na,na,false) current swing pivot low = pivot.new(na,na,false) current internal pivot high = pivot.new(na,na,false) current internal pivot low = pivot.new(na,na,false) current equal high pivot = pivot.new(na,na,false) current equal low pivot = pivot.new(na,na,false) swing trend bias = trend.new(0) internal trend bias = trend.new(0) equal high display = equalDisplay.new() equal low display = equalDisplay.new() storage for fairValueGap UDTs</pre>
--	---

```
var array<fairValueGap> fairValueGaps = array.new<fairValueGap>()
// @variable storage for parsed highs
var array<float> parsedHighs = array.new<float>()
// @variable storage for parsed lows
var array<float> parsedLows = array.new<float>()
// @variable storage for raw highs
var array<float> highs = array.new<float>()
// @variable storage for raw lows
var array<float> lows = array.new<float>()
// @variable storage for bar time values
var array<int> times = array.new<int>()
// @variable last trailing swing high and low
var trailingExtremes trailing = trailingExtremes.new()
// @variable storage for orderBlock
UDTs (swing order blocks)
var array<orderBlock> swingOrderBlocks =
array.new<orderBlock>()
// @variable storage for orderBlock
UDTs (internal order blocks)
var array<orderBlock> internalOrderBlocks =
array.new<orderBlock>()
// @variable storage for swing order
blocks boxes
var array<box> swingOrderBlocksBoxes = array.new<box>()
// @variable storage for internal
order blocks boxes
var array<box> internalOrderBlocksBoxes = array.new<box>()
// @variable color for swing bullish
structures
var swingBullishColor = styleInput == MONOCHROME ?
MONO_BULLISH : swingBullColorInput
// @variable color for swing bearish
structures
var swingBearishColor = styleInput == MONOCHROME ?
MONO_BEARISH : swingBearColorInput
// @variable color for bullish fair value
gaps
var fairValueGapBullishColor = styleInput == MONOCHROME ?
color.new(MONO_BULLISH,70) : fairValueGapsBullColorInput
// @variable color for bearish fair value
gaps
var fairValueGapBearishColor = styleInput == MONOCHROME ?
color.new(MONO_BEARISH,70) : fairValueGapsBearColorInput
// @variable color for premium zone
var premiumZoneColor = styleInput == MONOCHROME ?
MONO_BEARISH : premiumZoneColorInput
// @variable color for discount zone
var discountZoneColor = styleInput == MONOCHROME ?
MONO_BULLISH : discountZoneColorInput
// @variable bar index on current script
iteration
varip int currentBarIndex = bar_index
// @variable bar index on last script
iteration
```

```

varip int lastBarIndex          = bar_index
// @variable                    alerts in current bar
alerts currentAlerts           = alerts.new()
// @variable                    time at start of chart
var initialTime                = time

// we create the needed boxes for displaying order blocks at the
// first execution
if barstate.isfirst
    if showSwingOrderBlocksInput
        for index = 1 to swingOrderBlocksSizeInput
            swingOrderBlocksBoxes.push(box.new(na,na,na,na,xloc =
xloc.bar_time,extend = extend.right))
        if showInternalOrderBlocksInput
            for index = 1 to internalOrderBlocksSizeInput
                internalOrderBlocksBoxes.push(box.new(na,na,na,na,xloc =
xloc.bar_time,extend = extend.right))

// @variable                    source to use in bearish order
blocks mitigation
bearishOrderBlockMitigationSource = orderBlockMitigationInput ==
CLOSE ? close : high
// @variable                    source to use in bullish order
blocks mitigation
bullishOrderBlockMitigationSource = orderBlockMitigationInput ==
CLOSE ? close : low
// @variable                    default volatility measure
atrMeasure                     = ta.atr(200)
// @variable                    parsed volatility measure by
user settings
volatilityMeasure               = orderBlockFilterInput == ATR ?
atrMeasure : ta.cum(ta.tr)/bar_index
// @variable                    true if current bar is a high
volatility bar
highVolatilityBar               = (high - low) >= (2 *
volatilityMeasure)
// @variable                    parsed high
parsedHigh                      = highVolatilityBar ? low : high
// @variable                    parsed low
parsedLow                      = highVolatilityBar ? high : low

// we store current values into the arrays at each bar
parsedHighs.push(parsedHigh)
parsedLows.push(parsedLow)
highs.push(high)
lows.push(low)
times.push(time)

//-----
//-----}
//USER-DEFINED FUNCTIONS
//-----
//-----{
// @function                    Get the value of the current leg, it can be

```

```

0 (bearish) or 1 (bullish)
// @returns          int
leg(int size) =>
    var leg          = 0
    newLegHigh       = high[size] > ta.highest( size)
    newLegLow        = low[size]  < ta.lowest(  size)

    if newLegHigh
        leg := BEARISH_LEG
    else if newLegLow
        leg := BULLISH_LEG
    leg

// @function          Identify whether the current value is the
start of a new leg (swing)
// @param leg          (int) Current leg value
// @returns            bool
startOfNewLeg(int leg)    => ta.change(leg) != 0

// @function          Identify whether the current level is the
start of a new bearish leg (swing)
// @param leg          (int) Current leg value
// @returns            bool
startOfBearishLeg(int leg) => ta.change(leg) == -1

// @function          Identify whether the current level is the
start of a new bullish leg (swing)
// @param leg          (int) Current leg value
// @returns            bool
startOfBullishLeg(int leg) => ta.change(leg) == +1

// @function          create a new label
// @param labelTime    bar time coordinate
// @param labelPrice    price coordinate
// @param tag          text to display
// @param labelColor    text color
// @param labelStyle    label style
// @returns            label ID
drawLabel(int labelTime, float labelPrice, string tag, color
labelColor, string labelStyle) =>
    var label l_label = na

    if modeInput == PRESENT
        l_label.delete()

    l_label :=
label.new(chart.point.new(labelTime,na,labelPrice),tag,xloc.bar_time
,color=color(na),textcolor=labelColor,style = labelStyle,size =
size.small)

// @function          create a new line and label representing an
EQH or EQL
// @param p_pivot      starting pivot
// @param level         price level of current pivot

```

```

// @param size          how many bars ago was the current pivot
// detected
// @param equalHigh      true for EQH, false for EQL
// @returns              label ID
drawEqualHighLow(pivot p_ivot, float level, int size, bool
equalHigh) =>
    equalDisplay e_qualDisplay = equalHigh ? equalHighDisplay :
equalLowDisplay

    string tag          = 'EQL'
    color equalColor     = swingBullishColor
    string labelStyle    = label.style_label_up

    if equalHigh
        tag              := 'EQH'
        equalColor       := swingBearishColor
        labelStyle       := label.style_label_down

    if modeInput == PRESENT
        line.delete(     e_qualDisplay.l_line)
        label.delete(    e_qualDisplay.l_label)

        e_qualDisplay.l_line      :=
line.new(chart.point.new(p_ivot.barTime,na,p_ivot.currentLevel),
chart.point.new(time[size],na,level), xloc = xloc.bar_time, color =
equalColor, style = line.style_dotted)
        labelPosition            = math.round(0.5*(p_ivot.barIndex +
bar_index - size))
        e_qualDisplay.l_label    :=
label.new(chart.point.new(na,labelPosition,level), tag,
xloc.bar_index, color = color(na), textcolor = equalColor, style =
labelStyle, size = equalHighsLowsSizeInput)

// @function            store current structure and trailing swing
// points, and also display swing points and equal highs/ lows
// @param size          (int) structure size
// @param equalHighLow  (bool) true for displaying current highs/
// lows
// @param internal      (bool) true for getting internal structures
// @returns              label ID
getCurrentStructure(int size,bool equalHighLow = false, bool
internal = false) =>
    currentLeg          = leg(size)
    newPivot            = startOfNewLeg(currentLeg)
    pivotLow            = startOfBullishLeg(currentLeg)
    pivotHigh           = startOfBearishLeg(currentLeg)

    if newPivot
        if pivotLow
            pivot p_ivot    = equalHighLow ? equalLow : internal ?
internalLow : swingLow

            if equalHighLow and math.abs(p_ivot.currentLevel -
low[size]) < equalHighsLowsThresholdInput * atrMeasure

```

```

        drawEqualHighLow(p_ivot, low[size], size, false)
        currentAlerts.equalLows := true

        p_ivot.lastLevel      := p_ivot.currentLevel
        p_ivot.currentLevel := low[size]
        p_ivot.crossed       := false
        p_ivot.barTime       := time[size]
        p_ivot.barIndex      := bar_index[size]

        if not equalHighLow and not internal
            trailing.bottom      := p_ivot.currentLevel
            trailing.barTime     := p_ivot.barTime
            trailing.barIndex    := p_ivot.barIndex
            trailing.lastBottomTime := p_ivot.barTime

            if showSwingsInput and not internal and not equalHighLow
                drawLabel(time[size], p_ivot.currentLevel,
                    p_ivot.currentLevel < p_ivot.lastLevel ? 'LL' : 'HL',
                    swingBullishColor, label.style_label_up)
            else
                pivot p_ivot = equalHighLow ? equalHigh : internal ?
                internalHigh : swingHigh

                if equalHighLow and math.abs(p_ivot.currentLevel -
                    high[size]) < equalHighsLowsThresholdInput * atrMeasure
                    drawEqualHighLow(p_ivot, high[size], size, true)
                    currentAlerts.equalHighs := true

                p_ivot.lastLevel      := p_ivot.currentLevel
                p_ivot.currentLevel := high[size]
                p_ivot.crossed       := false
                p_ivot.barTime       := time[size]
                p_ivot.barIndex      := bar_index[size]

                if not equalHighLow and not internal
                    trailing.top      := p_ivot.currentLevel
                    trailing.barTime  := p_ivot.barTime
                    trailing.barIndex := p_ivot.barIndex
                    trailing.lastTopTime := p_ivot.barTime

                    if showSwingsInput and not internal and not equalHighLow
                        drawLabel(time[size], p_ivot.currentLevel,
                            p_ivot.currentLevel > p_ivot.lastLevel ? 'HH' : 'LH',
                            swingBearishColor, label.style_label_down)

// @function          draw line and label representing a
structure
// @param p_ivot      base pivot point
// @param tag          test to display
// @param structureColor base color
// @param lineStyle    line style
// @param labelStyle    label style
// @param labelSize    text size
// @returns            label ID

```

```

drawStructure(pivot p_pivot, string tag, color structureColor, string
lineStyle, string labelStyle, string labelSize) =>
    var line l_line      = line.new(na,na,na,na,xloc = xloc.bar_time)
    var label l_label    = label.new(na,na)

    if modeInput == PRESENT
        l_line.delete()
        l_label.delete()

    l_line :=
line.new(chart.point.new(p_pivot.barTime,na,p_pivot.currentLevel),
chart.point.new(time,na,p_pivot.currentLevel), xloc.bar_time,
color=structureColor, style=lineStyle)
    l_label :=
label.new(chart.point.new(na,math.round(0.5*(p_pivot.barIndex+bar_inde
x)),p_pivot.currentLevel), tag, xloc.bar_index, color=color(na),
textcolor=structureColor, style=labelStyle, size = labelSize)

// @function          delete order blocks
// @param internal    true for internal order blocks
// @returns           orderBlock ID
deleteOrderBlocks(bool internal = false) =>
    array<orderBlock> orderBlocks = internal ? internalOrderBlocks :
swingOrderBlocks

    for [index,eachOrderBlock] in orderBlocks
        bool crossedOrderBlock = false

        if bearishOrderBlockMitigationSource >
eachOrderBlock.barHigh and eachOrderBlock.bias == BEARISH
            crossedOrderBlock := true
            if internal
                currentAlerts.internalBearishOrderBlock := true
            else
                currentAlerts.swingBearishOrderBlock := true
        else if bullishOrderBlockMitigationSource <
eachOrderBlock.barLow and eachOrderBlock.bias == BULLISH
            crossedOrderBlock := true
            if internal
                currentAlerts.internalBullishOrderBlock := true
            else
                currentAlerts.swingBullishOrderBlock := true
        if crossedOrderBlock
            orderBlocks.remove(index)

// @function          fetch and store order blocks
// @param p_pivot      base pivot point
// @param internal    true for internal order blocks
// @param bias         BULLISH or BEARISH
// @returns           void
storeOrderBlock(pivot p_pivot,bool internal = false,int bias) =>
    if (not internal and showSwingOrderBlocksInput) or (internal and
showInternalOrderBlocksInput)

```

```

        array<float> a_rray = na
        int parsedIndex = na

        if bias == BEARISH
            a_rray :=
parsedHighs.slice(p_ivot.barIndex,bar_index)
            parsedIndex := p_ivot.barIndex +
a_rray.indexOf(a_rray.max())
        else
            a_rray :=
parsedLows.slice(p_ivot.barIndex,bar_index)
            parsedIndex := p_ivot.barIndex +
a_rray.indexOf(a_rray.min())

        orderBlock o_rderBlock =
orderBlock.new(parsedHighs.get(parsedIndex),
parsedLows.get(parsedIndex), times.get(parsedIndex),bias)
        array<orderBlock> orderBlocks = internal ?
internalOrderBlocks : swingOrderBlocks

        if orderBlocks.size() >= 100
            orderBlocks.pop()
            orderBlocks.unshift(o_rderBlock)

// @function          draw order blocks as boxes
// @param internal    true for internal order blocks
// @returns          void
drawOrderBlocks(bool internal = false) =>
    array<orderBlock> orderBlocks = internal ? internalOrderBlocks :
swingOrderBlocks
    orderBlocksSize = orderBlocks.size()

    if orderBlocksSize > 0
        maxOrderBlocks = internal ?
internalOrderBlocksSizeInput : swingOrderBlocksSizeInput
        array<orderBlock> parsedOrdeBlocks = orderBlocks.slice(0,
math.min(maxOrderBlocks,orderBlocksSize))
        array<box> b_oxes = internal ?
internalOrderBlocksBoxes : swingOrderBlocksBoxes

        for [index,eachOrderBlock] in parsedOrdeBlocks
            orderBlockColor = styleInput == MONOCHROME ?
(eachOrderBlock.bias == BEARISH ? color.new(MONO_BEARISH,80) :
color.new(MONO_BULLISH,80)) : internal ? (eachOrderBlock.bias ==
BEARISH ? internalBearishOrderBlockColor :
internalBullishOrderBlockColor) : (eachOrderBlock.bias == BEARISH ?
swingBearishOrderBlockColor : swingBullishOrderBlockColor)

            box b_ox = b_oxes.get(index)

b_ox.set_top_left_point(    chart.point.new(eachOrderBlock.barTime,n
a,eachOrderBlock.barHigh))

b_ox.set_bottom_right_point(chart.point.new(last_bar_time,na,eachOrd

```

```

erBlock.barLow))
        b_ox.set_border_color(      internal ? na :
orderBlockColor)
        b_ox.set_bgcolor(           orderBlockColor)

// @function          detect and draw structures, also detect and
store order blocks
// @param internal    true for internal structures or order blocks
// @returns           void
displayStructure(bool internal = false) =>
    var bullishBar = true
    var bearishBar = true

    if internalFilterConfluenceInput
        bullishBar := high - math.max(close, open) > math.min(close,
open - low)
        bearishBar := high - math.max(close, open) < math.min(close,
open - low)

    pivot p_ivot      = internal ? internalHigh : swingHigh
    trend t_rend      = internal ? internalTrend : swingTrend

    lineStyle          = internal ? line.style_dashed :
line.style_solid
    labelSize          = internal ? internalStructureSize :
swingStructureSize

    extraCondition     = internal ? internalHigh.currentLevel !=
swingHigh.currentLevel and bullishBar : true
    bullishColor       = styleInput == MONOCHROME ? MONO_BULLISH :
internal ? internalBullColorInput : swingBullColorInput

    if ta.crossover(close,p_ivot.currentLevel) and not
p_ivot.crossed and extraCondition
        string tag = t_rend.bias == BEARISH ? CHOCH : BOS

        if internal
            currentAlerts.internalBullishCHoCH := tag == CHOCH
            currentAlerts.internalBullishBOS   := tag == BOS
        else
            currentAlerts.swingBullishCHoCH    := tag == CHOCH
            currentAlerts.swingBullishBOS      := tag == BOS

        p_ivot.crossed := true
        t_rend.bias    := BULLISH

        displayCondition = internal ? showInternalsInput and
(showInternalBullInput == ALL or (showInternalBullInput == BOS and
tag != CHOCH) or (showInternalBullInput == CHOCH and tag ==
CHOCH)) : showStructureInput and (showSwingBullInput == ALL or
(showSwingBullInput == BOS and tag != CHOCH) or (showSwingBullInput
== CHOCH and tag == CHOCH))

        if displayCondition

```

```
drawStructure(p_ivot,tag,bullishColor,lineStyle,label.style_label_down,labelSize)
```

```
    if (internal and showInternalOrderBlocksInput) or (not  
internal and showSwingOrderBlocksInput)  
        storeOrderBlock(p_ivot,internal,BULLISH)
```

```
    p_ivot          := internal ? internalLow : swingLow  
    extraCondition  := internal ? internalLow.currentLevel !=  
swingLow.currentLevel and bearishBar : true  
    bearishColor    = styleInput == MONOCHROME ? MONO_BEARISH :  
internal ? internalBearColorInput : swingBearColorInput
```

```
    if ta.crossunder(close,p_ivot.currentLevel) and not  
p_ivot.crossed and extraCondition  
        string tag = t_rend.bias == BULLISH ? CHOCH : BOS
```

```
    if internal  
        currentAlerts.internalBearishCHOCH := tag == CHOCH  
        currentAlerts.internalBearishBOS   := tag == BOS  
    else  
        currentAlerts.swingBearishCHOCH    := tag == CHOCH  
        currentAlerts.swingBearishBOS      := tag == BOS
```

```
    p_ivot.crossed := true  
    t_rend.bias    := BEARISH
```

```
    displayCondition = internal ? showInternalsInput and  
(showInternalBearInput == ALL or (showInternalBearInput == BOS and  
tag != CHOCH) or (showInternalBearInput == CHOCH and tag ==  
CHOCH)) : showStructureInput and (showSwingBearInput == ALL or  
(showSwingBearInput == BOS and tag != CHOCH) or (showSwingBearInput  
== CHOCH and tag == CHOCH))
```

```
    if displayCondition
```

```
drawStructure(p_ivot,tag,bearishColor,lineStyle,label.style_label_up  
,labelSize)
```

```
    if (internal and showInternalOrderBlocksInput) or (not  
internal and showSwingOrderBlocksInput)  
        storeOrderBlock(p_ivot,internal,BEARISH)
```

```
// @function          draw one fair value gap box (each fair value  
gap has two boxes)  
// @param leftTime    left time coordinate  
// @param rightTime   right time coordinate  
// @param topPrice     top price level  
// @param bottomPrice bottom price level  
// @param boxColor     box color  
// @returns           box ID  
fairValueGapBox(leftTime,rightTime,topPrice,bottomPrice,boxColor) =>  
box.new(chart.point.new(leftTime,na,topPrice),chart.point.new(rightT
```

```

ime + fairValueGapsExtendInput * (time-time[1]),na,bottomPrice),
xloc=xloc.bar_time, border_color = boxColor, bgcolor = boxColor)

// @function          delete fair value gaps
// @returns            fairValueGap ID
deleteFairValueGaps() =>
    for [index,eachFairValueGap] in fairValueGaps
        if (low < eachFairValueGap.bottom and eachFairValueGap.bias
== BULLISH) or (high > eachFairValueGap.top and
eachFairValueGap.bias == BEARISH)
            eachFairValueGap.topBox.delete()
            eachFairValueGap.bottomBox.delete()
            fairValueGaps.remove(index)

// @function          draw fair value gaps
// @returns            fairValueGap ID
drawFairValueGaps() =>
    [lastClose, lastOpen, lastTime, currentHigh, currentLow,
currentTime, last2High, last2Low] =
request.security(syminfo.tickerid, fairValueGapsTimeframeInput,
[close[1], open[1], time[1], high[0], low[0], time[0], high[2],
low[2]],lookahead = barmerge.lookahead_on)

    barDeltaPercent      = (lastClose - lastOpen) / (lastOpen * 100)
    newTimeframe          =
timeframe.change(fairValueGapsTimeframeInput)
    threshold             = fairValueGapsThresholdInput ?
ta.cum(math.abs(newTimeframe ? barDeltaPercent : 0)) / bar_index *
2 : 0

    bullishFairValueGap = currentLow > last2High and lastClose >
last2High and barDeltaPercent > threshold and newTimeframe
    bearishFairValueGap = currentHigh < last2Low and lastClose <
last2Low and -barDeltaPercent > threshold and newTimeframe

    if bullishFairValueGap
        currentAlerts.bullishFairValueGap := true

fairValueGaps.unshift(fairValueGap.new(currentLow,last2High,BULLISH,
fairValueGapBox(lastTime,currentTime,currentLow,math.avg(currentLow,
last2High),fairValueGapBullishColor),fairValueGapBox(lastTime,curren
tTime,math.avg(currentLow,last2High),last2High,fairValueGapBullishCo
lor)))
    if bearishFairValueGap
        currentAlerts.bearishFairValueGap := true

fairValueGaps.unshift(fairValueGap.new(currentHigh,last2Low,BEARISH,
fairValueGapBox(lastTime,currentTime,currentHigh,math.avg(currentHig
h,last2Low),fairValueGapBearishColor),fairValueGapBox(lastTime,curren
tTime,math.avg(currentHigh,last2Low),last2Low,fairValueGapBearishCo
lor)))

// @function          get line style from string
// @param style        line style

```

```

// @returns          string
getStyle(string style) =>
    switch style
        SOLID => line.style_solid
        DASHED => line.style_dashed
        DOTTED => line.style_dotted

// @function          draw MultiTimeFrame levels
// @param timeframe    base timeframe
// @param sameTimeframe true if chart timeframe is same as base
// @param style         line style
// @param levelColor     line and text color
// @returns            void
drawLevels(string timeframe, bool sameTimeframe, string style, color
levelColor) =>
    [topLevel, bottomLevel, leftTime, rightTime] =
request.security(syminfo.tickerid, timeframe, [high[1], low[1],
time[1], time], lookahead = barmerge.lookahead_on)

    float parsedTop          = sameTimeframe ? high : topLevel
    float parsedBottom        = sameTimeframe ? low : bottomLevel

    int parsedLeftTime        = sameTimeframe ? time : leftTime
    int parsedRightTime        = sameTimeframe ? time : rightTime

    int parsedTopTime          = time
    int parsedBottomTime        = time

    if not sameTimeframe
        int leftIndex          =
times.binary_search_rightmost(parsedLeftTime)
        int rightIndex          =
times.binary_search_rightmost(parsedRightTime)

        array<int> timeArray      =
times.slice(leftIndex, rightIndex)
        array<float> topArray      =
highs.slice(leftIndex, rightIndex)
        array<float> bottomArray    =
lows.slice(leftIndex, rightIndex)

        parsedTopTime             := timeArray.size() > 0 ?
timeArray.get(topArray.indexof(topArray.max())) : initialTime
        parsedBottomTime           := timeArray.size() > 0 ?
timeArray.get(bottomArray.indexof(bottomArray.min())) : initialTime

    var line topLine            = line.new(na, na, na, na, xloc =
xloc.bar_time, color = levelColor, style = getStyle(style))
    var line bottomLine         = line.new(na, na, na, na, xloc =
xloc.bar_time, color = levelColor, style = getStyle(style))
    var label topLabel          = label.new(na, na, xloc =
xloc.bar_time, text = str.format('P{0}H', timeframe),
color=color(na), textcolor = levelColor, size = size.small, style =

```

```

label.style_label_left)
    var label bottomLabel = label.new(na, na, xloc =
xloc.bar_time, text = str.format('P{0}L',timeframe),
color=color(na), textcolor = levelColor, size = size.small, style =
label.style_label_left)

topLine.set_first_point(    chart.point.new(parsedTopTime,na,parsedT
op))
    topLine.set_second_point(    chart.point.new(last_bar_time + 20 *
(time-time[1]),na,parsedTop))
    topLabel.set_point(        chart.point.new(last_bar_time + 20 *
(time-time[1]),na,parsedTop))

bottomLine.set_first_point( chart.point.new(parsedBottomTime,na,pars
edBottom))
    bottomLine.set_second_point(chart.point.new(last_bar_time + 20 *
(time-time[1]),na,parsedBottom))
    bottomLabel.set_point(        chart.point.new(last_bar_time + 20 *
(time-time[1]),na,parsedBottom))

// @function            true if chart timeframe is higher than
provided timeframe
// @param timeframe     timeframe to check
// @returns              bool
higherTimeframe(string timeframe) => timeframe.in_seconds() >
timeframe.in_seconds(timeframe)

// @function            update trailing swing points
// @returns              int
updateTrailingExtremes() =>
    trailing.top           := math.max(high,trailing.top)
    trailing.lastTopTime   := trailing.top == high ? time :
trailing.lastTopTime
    trailing.bottom        := math.min(low,trailing.bottom)
    trailing.lastBottomTime := trailing.bottom == low ? time :
trailing.lastBottomTime

// @function            draw trailing swing points
// @returns              void
drawHighLowSwings() =>
    var line topLine       = line.new(na, na, na, na, color =
swingBearishColor, xloc = xloc.bar_time)
    var line bottomLine    = line.new(na, na, na, na, color =
swingBullishColor, xloc = xloc.bar_time)
    var label topLabel     = label.new(na, na, color=color(na),
textcolor = swingBearishColor, xloc = xloc.bar_time, style =
label.style_label_down, size = size.tiny)
    var label bottomLabel  = label.new(na, na, color=color(na),
textcolor = swingBullishColor, xloc = xloc.bar_time, style =
label.style_label_up, size = size.tiny)

    rightTimeBar           = last_bar_time + 20 * (time - time[1])

```

```

topLine.set_first_point(    chart.point.new(trailing.lastTopTime,
na, trailing.top))
    topLine.set_second_point(    chart.point.new(rightTimeBar, na,
trailing.top))
    topLabel.set_point(        chart.point.new(rightTimeBar, na,
trailing.top))
    topLabel.set_text(        swingTrend.bias == BEARISH ? 'Strong
High' : 'Weak High')

```

```

bottomLine.set_first_point( chart.point.new(trailing.lastBottomTime,
na, trailing.bottom))
    bottomLine.set_second_point(chart.point.new(rightTimeBar, na,
trailing.bottom))
    bottomLabel.set_point(    chart.point.new(rightTimeBar, na,
trailing.bottom))
    bottomLabel.set_text(    swingTrend.bias == BULLISH ? 'Strong
Low' : 'Weak Low')

```

```

// @function            draw a zone with a label and a box
// @param labelLevel    price level for label
// @param labelIndex    bar index for label
// @param top           top price level for box
// @param bottom        bottom price level for box
// @param tag           text to display
// @param zoneColor     base color
// @param style         label style
// @returns             void
drawZone(float labelLevel, int labelIndex, float top, float bottom,
string tag, color zoneColor, string style) =>
    var label l_label    = label.new(na,na,text = tag,
color=color(na),textcolor = zoneColor, style = style, size =
size.small)
    var box b_ox         = box.new(na,na,na,na,bgcolor =
color.new(zoneColor,80),border_color = color(na), xloc =
xloc.bar_time)

```

```

b_ox.set_top_left_point(    chart.point.new(trailing.barTime,na,top)
)

```

```

b_ox.set_bottom_right_point(chart.point.new(last_bar_time,na,bottom)
)

```

```

l_label.set_point(        chart.point.new(na,labelIndex,labelLevel
))

```

```

// @function            draw premium/discount zones
// @returns             void
drawPremiumDiscountZones() =>
    drawZone(trailing.top, math.round(0.5*(trailing.barIndex +

```

```

last_bar_index)), trailing.top, 0.95*trailing.top +
0.05*trailing.bottom, 'Premium', premiumZoneColor,
label.style_label_down)

    equilibriumLevel = math.avg(trailing.top, trailing.bottom)
    drawZone(equilibriumLevel, last_bar_index, 0.525*trailing.top +
0.475*trailing.bottom, 0.525*trailing.bottom + 0.475*trailing.top,
'Equilibrium', equilibriumZoneColorInput, label.style_label_left)

    drawZone(trailing.bottom, math.round(0.5*(trailing.barIndex +
last_bar_index)), 0.95*trailing.bottom + 0.05*trailing.top,
trailing.bottom, 'Discount', discountZoneColor,
label.style_label_up)

//-----
-----}
//MUTABLE VARIABLES & EXECUTION
//-----
-----{
parsedOpen = showTrendInput ? open : na
candleColor = internalTrend.bias == BULLISH ? swingBullishColor :
swingBearishColor
plotcandle(parsedOpen,high,low,close,color = candleColor, wickcolor
= candleColor, bordercolor = candleColor)

if showHighLowSwingsInput or showPremiumDiscountZonesInput
    updateTrailingExtremes()

    if showHighLowSwingsInput
        drawHighLowSwings()

    if showPremiumDiscountZonesInput
        drawPremiumDiscountZones()

if showFairValueGapsInput
    deleteFairValueGaps()

getCurrentStructure(swingsLengthInput,false)
getCurrentStructure(5,false,true)

if showEqualHighsLowsInput
    getCurrentStructure(equalHighsLowsLengthInput,true)

if showInternalsInput or showInternalOrderBlocksInput or
showTrendInput
    displayStructure(true)

if showStructureInput or showSwingOrderBlocksInput or
showHighLowSwingsInput
    displayStructure()

if showInternalOrderBlocksInput
    deleteOrderBlocks(true)

```

```

if showSwingOrderBlocksInput
    deleteOrderBlocks()

if showFairValueGapsInput
    drawFairValueGaps()

if barstate.islastconfirmedhistory or barstate.islast
    if showInternalOrderBlocksInput
        drawOrderBlocks(true)

        if showSwingOrderBlocksInput
            drawOrderBlocks()

lastBarIndex    := currentBarIndex
currentBarIndex := bar_index
newBar          = currentBarIndex != lastBarIndex

if barstate.islastconfirmedhistory or (barstate.isrealtime and
newBar)
    if showDailyLevelsInput and not higherTimeframe('D')

drawLevels('D', timeframe.isdaily, dailyLevelsStyleInput, dailyLevelsCo
lorInput)

    if showWeeklyLevelsInput and not higherTimeframe('W')

drawLevels('W', timeframe.isweekly, weeklyLevelsStyleInput, weeklyLevel
sColorInput)

    if showMonthlyLevelsInput and not higherTimeframe('M')

drawLevels('M', timeframe.ismonthly, monthlyLevelsStyleInput, monthlyLe
velsColorInput)

//-----
//ALERTS
//-----
-----{
alertcondition(currentAlerts.internalBullishBOS,      'Internal
Bullish BOS',      'Internal Bullish BOS formed')
alertcondition(currentAlerts.internalBullishCHoCH,    'Internal
Bullish CHoCH',    'Internal Bullish CHoCH formed')
alertcondition(currentAlerts.internalBearishBOS,      'Internal
Bearish BOS',      'Internal Bearish BOS formed')
alertcondition(currentAlerts.internalBearishCHoCH,    'Internal
Bearish CHoCH',    'Internal Bearish CHoCH formed')

alertcondition(currentAlerts.swingBullishBOS,         'Bullish
BOS',         'Internal Bullish BOS formed')
alertcondition(currentAlerts.swingBullishCHoCH,       'Bullish
CHoCH',       'Internal Bullish CHoCH formed')
alertcondition(currentAlerts.swingBearishBOS,         'Bearish
BOS',         'Bearish BOS formed')

```

```
alertcondition(currentAlerts.swingBearishCHoCH, 'Bearish  
CHoCH', 'Bearish CHoCH formed')
```

```
alertcondition(currentAlerts.internalBullishOrderBlock, 'Bullish  
Internal OB Breakout', 'Price broke bullish internal OB')
```

```
alertcondition(currentAlerts.internalBearishOrderBlock, 'Bearish  
Internal OB Breakout', 'Price broke bearish internal OB')
```

```
alertcondition(currentAlerts.swingBullishOrderBlock, 'Bullish  
Swing OB Breakout', 'Price broke bullish swing OB')
```

```
alertcondition(currentAlerts.swingBearishOrderBlock, 'Bearish  
Swing OB Breakout', 'Price broke bearish swing OB')
```

```
alertcondition(currentAlerts.equalHighs, 'Equal  
Highs', 'Equal highs detected')
```

```
alertcondition(currentAlerts.equalLows, 'Equal  
Lows', 'Equal lows detected')
```

```
alertcondition(currentAlerts.bullishFairValueGap, 'Bullish  
FVG', 'Bullish FVG formed')
```

```
alertcondition(currentAlerts.bearishFairValueGap, 'Bearish  
FVG', 'Bearish FVG formed')
```

```
//-----  
-----}
```